

MultiAncestryPGxGWAS: Simulation and Application

Wujuan Zhong, Judong Shen

2026-10-05

Overview

MultiAncestryPGxGWAS is an R package to conduct genome-wide genotype-by-environment (GxE) or genotype-by-treatment interaction (GxT) analysis powered by the following methods: Linear regression with robust (sandwich) variance,

This vignette demonstrates how to conduct genome-wide GxT analysis using an example genotype data in PLINK v1 binary format (bed/bim/fam).

- Generate or load example genotype data in PLINK v1 binary format.
- Then separately for continuous and binary outcomes,
 - Simulate phenotypes and covariates.
 - Run genome-wide GxT association.
- Summarize the results.

The preparation and simulation examples use a simulated genotype data; association analyses require the external tools listed below.

- PLINK1.9 (<https://www.cog-genomics.org/plink/1.9/>): v1.90b5.1
- PLINK2 (<https://www.cog-genomics.org/plink/2.0/>): v2.00a2LM
- GCTA (<https://yanglab.westlake.edu.cn/software/gcta/#Download>): v1.95.3
- LEMMA (<https://github.com/mkerin/LEMMA.git>): 1.0.2+
- GEM (<https://github.com/large-scale-gxe-methods/GEM.git>): 1.4.5+
- bgenix (<https://enkre.net/cgi-bin/code/bgen/dir?ci=trunk>): 1.1.7
- MR-MEGA (<https://genomics.ut.ee/en/tools>): 0.2
- (optional) mpirun (<https://docs.open-mpi.org/en/main/index.html>): for MPI-based LEMMA runs: 4.1.7

Apply MultiAncestryPGxGWAS to a Continuous Outcome and a Binary Outcome

- Continuous outcome methods: `fastGWA_GE`, `LEMMA`, `GENESIS`, `LR-Sandwich`, fixed-effects meta-analysis of `LR-Sandwich`, and `MR-MEGA` of `LR-Sandwich`.
- Binary outcome methods: `fastGWA_GE`, `LEMMA`, `GENESIS`, `GEM`, and `MR-MEGA` of `GEM`. Note: `fastGWA_GE` and `LEMMA` naively treat the binary outcome as 0/1.

These examples require external tools (`plink`, `plink2`, `flashpca`, `gcta`, `lemma_1_0_4`, `GEM`, `bgenix`, `MR-MEGA`) plus `GENESIS` R package.

We first set up paths to external tools and working directories.

```
library(MultiAncestryPGxGWAS)
```

```

# If your site does not expose the command names below,
# define `configs` with explicit executable paths.
configs <- list(
  plink1.9.path = "plink", # path to plink
  plink2.path = "plink2", # path to plink2
  flashpca.path = "flashpca_x86-64", # path to flashpca
  gcta.path = "gcta", # path to gcta
  lemma.path = "lemma_1_0_4", # path to lemma
  gem.path = "GEM_1.4.5_static", # path to GEM
  bgenix.path = "bgenix", # path to bgenix
  mrmega.path = "MR-MEGA", # path to MR-MEGA
  mpirun.path = "mpirun" # path to mpirun
)

mpgx_validate_tool_configs(configs)
#> OK

work_dir <- file.path(tempdir(), "MultiAncestryPGxGWAS_vignette")

methods_output_dir <- file.path(work_dir, "model_results")
dir.create(methods_output_dir, recursive = TRUE, showWarnings = FALSE)

method_out_dirs <- list(
  genotype_artifacts = file.path(work_dir, "genotype_artifacts"),
  continuous = list(
    root = file.path(methods_output_dir, "continuous"),
    lr_sandwich = file.path(methods_output_dir, "continuous", "lr_sandwich"),
    fe_meta_lr = file.path(methods_output_dir, "continuous", "fe_meta_lr"),
    mrmega_lr = file.path(methods_output_dir, "continuous", "mrmega_lr"),
    fastgwa_ge = file.path(methods_output_dir, "continuous", "fastgwa_ge"),
    genesis = file.path(methods_output_dir, "continuous", "genesis"),
    lemma = file.path(methods_output_dir, "continuous", "lemma")
  ),
  binary = list(
    root = file.path(methods_output_dir, "binary"),
    fastgwa_ge_naive = file.path(methods_output_dir, "binary", "fastgwa_ge_naive"),
    lemma_naive = file.path(methods_output_dir, "binary", "lemma_naive"),
    genesis = file.path(methods_output_dir, "binary", "genesis"),
    gem = file.path(methods_output_dir, "binary", "gem"),
    mrmega_gem = file.path(methods_output_dir, "binary", "mrmega_gem")
  )
)

method_leaf_dirs <- unique(c(
  method_out_dirs$genotype_artifacts,
  unlist(method_out_dirs$continuous, use.names = FALSE),
  unlist(method_out_dirs$binary, use.names = FALSE)
))
invisible(lapply(method_leaf_dirs, dir.create, recursive = TRUE, showWarnings = FALSE))

```

Prepare a small example genotype dataset used by all methods below.

We use package helper `mpgx_prepare_ancestry_keep_files()` for ancestry-specific keep files.

```

prepare_geno <- mpgx_prepare_example_genotype(
  output_dir = file.path(work_dir, "genotype_data"),
  prefix = "toyData",
  source = "toy_data",
  configs = configs,
  overwrite = TRUE,
  pcs_output_dir = method_out_dirs$genotype_artifacts
)

genotype_artifacts <- mpgx_prepare_genotype_artifacts(
  genotype_prefix_or_path = prepare_geno$prefix,
  out_dir = method_out_dirs$genotype_artifacts,
  configs = configs
)

simulation_seed <- 100001

```

Simulate one continuous-outcome run and prepare run-specific input files, reusing genotype artifacts.

```

cont <- mpgx_simulate_phenotypes(
  bfile_prefix = prepare_geno$prefix,
  ancestry_info_file = prepare_geno$ancestry_info_file,
  output_dir = file.path(work_dir, "simulated_data"),
  result_label = "continuous",
  seed = simulation_seed,
  num_pcs = 10,
  pcs_file = prepare_geno$pcs_file,
  variance_proportion = "0.25_0.25_0.05_0.05_0.00_0.40",
  n_snps_maineffects_only = 2,
  n_snps_interaction_only = 2,
  n_snps_both = 6,
  effect_size_mean = 6,
  type_of_phenotype = "normal",
  configs = configs
)

cont_keep <- mpgx_prepare_ancestry_keep_files(
  ancestry_info_file = prepare_geno$ancestry_info_file,
  out_dir = method_out_dirs$continuous$root
)

cont_inputs <- list(
  bfile_prefix = prepare_geno$prefix,
  seed = cont$seed,
  result_label = cont$result_label,
  result_dir = cont$result_dir,
  phenotype_file = cont$phenotype_file,
  qcovar_file = cont$qcovar_file,
  covar_file = cont$covar_file,
  envir_file = cont$envir_file,
  sparse_grm_prefix = genotype_artifacts$sparse_grm_prefix,
  bgen_file = genotype_artifacts$bgen_file,

```

```

gds_file = genotype_artifacts$gds_file,
keep_files = cont_keep$keep_files,
lemma_pheno = file.path(cont$result_dir,
  sprintf("sim_phenotype_forLEMMMA_seed_%d.txt", cont$seed)),
lemma_envir = file.path(cont$result_dir, "sim_envir_forLEMMMA.txt"),
lemma_covar = file.path(cont$result_dir, "sim_covar_qcovar_forLEMMMA.txt"),
gem_pheno = file.path(cont$result_dir,
  sprintf("sim_phenotype_forGEM_seed_%d.txt", cont$seed))
)

```

Run continuous-outcome methods.

Each method is separated into its own chunk and stores a parsed GWAS data frame in each requested output object.

We first run LR-Sandwich on all samples and then per ancestry to create inputs for meta-analysis methods.

```

# LR-Sandwich on all samples.
lr_sandwich_all_prefix <- file.path(
  method_out_dirs$continuous$lr_sandwich, "continuous.LRallSandwich")
lr_sandwich_all <- mpgx_run_lr_sandwich(
  bfile_prefix = cont_inputs$bfile_prefix,
  phenotype_file = cont_inputs$phenotype_file,
  qcovar_file = cont_inputs$qcovar_file,
  covar_file = cont_inputs$covar_file,
  envir_file = cont_inputs$envir_file,
  output_prefix = lr_sandwich_all_prefix,
  sandwich = TRUE,
  configs = configs
)

# LR-Sandwich per ancestry (inputs for fixed-effects meta and MR-MEGA).
ancestry_levels <- names(cont_inputs$keep_files)
lr_sandwich_by_eth <- lapply(ancestry_levels, function(ances) {
  out_prefix <- file.path(
    method_out_dirs$continuous$lr_sandwich, paste0("continuous.LRallSandwich.", ances))
  mpgx_run_lr_sandwich(
    bfile_prefix = cont_inputs$bfile_prefix,
    phenotype_file = cont_inputs$phenotype_file,
    qcovar_file = cont_inputs$qcovar_file,
    covar_file = cont_inputs$covar_file,
    envir_file = cont_inputs$envir_file,
    keep_file = cont_inputs$keep_files[[ances]],
    output_prefix = out_prefix,
    sandwich = TRUE,
    configs = configs
  )
})
names(lr_sandwich_by_eth) <- ancestry_levels
lr_sandwich_by_eth_files <- extract_gwas_files(lr_sandwich_by_eth)

```

We then combine per-ancestry LR-Sandwich outputs using fixed-effect meta-analysis and MR-MEGA.

```
# Fixed-effects meta-analysis of LR-Sandwich.
meta_lr_cont_file <- file.path(
  method_out_dirs$continuous$fe_meta_lr, "continuous.meta_LRallSandwich.txt")
meta_lr_cont <- mpgx_run_fe_meta_for_lr(
  ancestry_result_files = lr_sandwich_by_eth_files,
  output_file = meta_lr_cont_file,
  ancestry_labels = ancestry_levels
)

# MR-MEGA of LR-Sandwich.
mrmega_lr_cont <- mpgx_run_mrmega_for_lr(
  ancestry_result_files = lr_sandwich_by_eth_files,
  output_prefix_base = file.path(
    method_out_dirs$continuous$mrmega_lr, "continuous.MRMEGA_LRallSandwich"),
  configs = configs
)
```

Next, run fastGWA_GE for the continuous phenotype using the precomputed sparse GRM.

```
# fastGWA_GE (continuous).
fastgwa_ge_cont_prefix <- file.path(
  method_out_dirs$continuous$fastgwa_ge, "continuous.fastGWA_GE")
fastgwa_ge_cont <- mpgx_run_fastgwa_ge(
  bfile_prefix = cont_inputs$bfile_prefix,
  grm_sparse_prefix = cont_inputs$sparse_grm_prefix,
  phenotype_file = cont_inputs$phenotype_file,
  qcovar_file = cont_inputs$qcovar_file,
  covar_file = cont_inputs$covar_file,
  envir_file = cont_inputs$envir_file,
  output_prefix = fastgwa_ge_cont_prefix,
  sandwich = TRUE,
  configs = configs
)
```

Then run GENESIS on the same continuous-outcome inputs for comparison.

```
# GENESIS (continuous).
genesis_cont_file <- file.path(
  method_out_dirs$continuous$genesis, "continuous.GENESIS.txt")
genesis_cont <- mpgx_run_genesis(
  gds_file = cont_inputs$gds_file,
  sparse_grm_prefix = cont_inputs$sparse_grm_prefix,
  phenotype_file = cont_inputs$phenotype_file,
  qcovar_file = cont_inputs$qcovar_file,
  covar_file = cont_inputs$covar_file,
  envir_file = cont_inputs$envir_file,
  output_file = genesis_cont_file
)
```

Finally, run LEMMA for continuous outcome analysis with the LEMMA-formatted inputs.

```
# LEMMA (continuous).
lemma_cont_prefix <- file.path(method_out_dirs$continuous$lemma, "continuous.LEMMA")
lemma_cont <- mpgx_run_lemma(
  bgen_file = cont_inputs$bgen_file,
  phenotype_file = cont_inputs$lemma_pheno,
  enviro_file = cont_inputs$lemma_envir,
  covar_file = cont_inputs$lemma_covar,
  output_prefix = lemma_cont_prefix,
  n_tasks = 1,
  configs = configs
)
```

Simulate one binary-outcome run and prepare run-specific local inputs, reusing genotype artifacts.

```
binary <- mpgx_simulate_phenotypes(
  bfile_prefix = prepare_geno$prefix,
  ancestry_info_file = prepare_geno$ancestry_info_file,
  output_dir = file.path(work_dir, "simulated_data"),
  result_label = "binary",
  seed = simulation_seed,
  num_pcs = 10,
  pcs_file = prepare_geno$pcs_file,
  variance_proportion = "0.25_0.25_0.05_0.05_0.00_0.40",
  n_snps_maineffects_only = 2,
  n_snps_interaction_only = 2,
  n_snps_both = 6,
  effect_size_mean = 6,
  type_of_phenotype = "binary",
  configs = configs
)

bin_keep <- mpgx_prepare_ancestry_keep_files(
  ancestry_info_file = prepare_geno$ancestry_info_file,
  out_dir = method_out_dirs$binary$root
)

bin_inputs <- list(
  bfile_prefix = prepare_geno$prefix,
  seed = binary$seed,
  result_label = binary$result_label,
  result_dir = binary$result_dir,
  phenotype_file = binary$phenotype_file,
  qcovar_file = binary$qcovar_file,
  covar_file = binary$covar_file,
  enviro_file = binary$enviro_file,
  sparse_grm_prefix = genotype_artifacts$sparse_grm_prefix,
  bgen_file = genotype_artifacts$bgen_file,
  gds_file = genotype_artifacts$gds_file,
  keep_files = bin_keep$keep_files,
  lemma_pheno = file.path(
```

```

    binary$result_dir, sprintf("sim_phenotype_forLEMMMA_seed_%d.txt", binary$seed)),
  lemma_envir = file.path(binary$result_dir, "sim_envir_forLEMMMA.txt"),
  lemma_covar = file.path(binary$result_dir, "sim_covar_qcovar_forLEMMMA.txt"),
  gem_pheno = file.path(
    binary$result_dir, sprintf("sim_phenotype_forGEM_seed_%d.txt", binary$seed))
)

```

Run binary-outcome methods in separate chunks, including per-ancestry GEM preparation for MR-MEGA.

We first run fastGWA_GE while naively treating the binary phenotype as 0/1.

```

# Naively treat binary phenotype as 0/1 for fastGWA_GE.
fastgwa_bin_naive_prefix <- file.path(
  method_out_dirs$binary$fastgwa_ge_naive, "binary.fastGWA_GE_naive")
fastgwa_bin_naive <- mpgx_run_fastgwa_ge(
  bfile_prefix = bin_inputs$bfile_prefix,
  grm_sparse_prefix = bin_inputs$sparse_grm_prefix,
  phenotype_file = bin_inputs$phenotype_file,
  qcovar_file = bin_inputs$qcovar_file,
  covar_file = bin_inputs$covar_file,
  envir_file = bin_inputs$envir_file,
  output_prefix = fastgwa_bin_naive_prefix,
  sandwich = TRUE,
  configs = configs
)

```

Next, run LEMMA with the same naive binary encoding and LEMMA-specific inputs.

```

# Naively treat binary phenotype as 0/1 for LEMMA.
lemma_bin_naive_prefix <- file.path(
  method_out_dirs$binary$lemma_naive, "binary.LEMMA_naive")
lemma_bin_naive <- mpgx_run_lemma(
  bgen_file = bin_inputs$bgen_file,
  phenotype_file = bin_inputs$lemma_pheno,
  envir_file = bin_inputs$lemma_envir,
  covar_file = bin_inputs$lemma_covar,
  output_prefix = lemma_bin_naive_prefix,
  configs = configs
)

```

Then run GENESIS for binary-outcome association testing using the GDS input.

```

genesis_bin_file <- file.path(method_out_dirs$binary$genesis, "binary.GENESIS.txt")
genesis_bin <- mpgx_run_genesis(
  gds_file = bin_inputs$gds_file,
  sparse_grm_prefix = bin_inputs$sparse_grm_prefix,
  phenotype_file = bin_inputs$phenotype_file,
  qcovar_file = bin_inputs$qcovar_file,
  covar_file = bin_inputs$covar_file,
  envir_file = bin_inputs$envir_file,
  output_file = genesis_bin_file
)

```

After that, run GEM on the full sample to produce a binary-outcome GWAS result.

```
gem_bin_prefix <- file.path(method_out_dirs$binary$gem, "binary.GEM")
gem_bin <- mpgx_run_gem(
  bfile_prefix = bin_inputs$bfile_prefix,
  phenotype_file = bin_inputs$gem_pheno,
  output_prefix = gem_bin_prefix,
  configs = configs
)
```

Finally, prepare per-ancestry GEM outputs and run MR-MEGA on those ancestry-specific results.

```
# Per-ancestry GEM inputs and runs for MR-MEGA.
gem_by_eth_files <- mpgx_prepare_mrmega_gem_inputs(
  bfile_prefix = bin_inputs$bfile_prefix,
  gem_phenotype_file = bin_inputs$gem_pheno,
  keep_files = bin_inputs$keep_files,
  output_dir = method_out_dirs$binary$gem,
  configs = configs
)

# MR-MEGA of GEM (binary, OR-scale input).
pc_num_bin <- max(length(gem_by_eth_files) - 3, 0)
mrmega_gem <- mpgx_run_mrmega_for_gem(
  ancestry_result_files = gem_by_eth_files,
  output_prefix_base = file.path(method_out_dirs$binary$mrmega_gem, "binary.MRMEGA_GEM"),
  configs = configs
)
```

Summarize Parsed GWAS Results

This chunk applies `mpgx_summarize_association()` to each method's parsed GWAS data frame (`$gwas_df`) for both continuous and binary outcomes.

```
summarize_method_list <- function(
  method_results,
  causal_snps,
  scenario,
  seed,
  mrmega_methods = character(),
  mrmega_pc_num = NULL
) {
  summaries <- lapply(names(method_results), function(method_name) {
    method_df <- mpgx_get_method_gwas_df(method_results[[method_name]], label = method_name)
    mrmega_pc <- if (method_name %in% mrmega_methods) mrmega_pc_num else NULL
    # Keep rows when a method has no 2-df test (for example MR-MEGA or LEMMA).
    has_non_missing_2df <- "P_2df" %in% colnames(method_df) && any(!is.na(method_df$P_2df))
    summary_df <- mpgx_summarize_association(
      result = method_df,
      causal_snps = causal_snps,

```

```

    rm_na_2df = has_non_missing_2df,
    mrmega_pc_num = mrmega_pc
  )

  cbind(
    data.frame(
      method = method_name,
      scenario = scenario,
      seed = seed,
      stringsAsFactors = FALSE
    ),
    summary_df
  )
})

out <- do.call(rbind, summaries)
rownames(out) <- NULL
out
}

continuous_methods <- list(
  LRallSandwich = lr_sandwich_all,
  meta_LRallSandwich = meta_lr_cont,
  MRMEGA_LRallSandwich = mrmega_lr_cont,
  fastGWA_GE = fastgwa_ge_cont,
  GENESIS = genesis_cont,
  LEMMA = lemma_cont
)

continuous_summary <- summarize_method_list(
  method_results = continuous_methods,
  causal_snps = union(cont$causal_main_snps, cont$causal_interaction_snps),
  scenario = cont$result_label,
  seed = cont_inputs$seed,
  mrmega_methods = "MRMEGA_LRallSandwich",
  mrmega_pc_num = max(length(lr_sandwich_by_eth_files) - 3, 0)
)

continuous_summary
#>           method    scenario    seed genomic_inflation_factor_G
#> 1      LRallSandwich continuous 100001          0.9680583
#> 2    meta_LRallSandwich continuous 100001          1.1550683
#> 3 MRMEGA_LRallSandwich continuous 100001          1.0316640
#> 4      fastGWA_GE continuous 100001          0.9861246
#> 5          GENESIS continuous 100001          0.7322637
#> 6          LEMMA continuous 100001          0.9770279
#> genomic_inflation_factor_G_by_E genomic_inflation_factor_2df    FPR_G
#> 1                0.9469323          1.003596 0.04688547
#> 2                1.1535661          1.363771 0.07970529
#> 3                1.1394478                NA 0.08573342
#> 4                0.9324358          1.019834 0.04755526
#> 5                0.8494997          0.942750 0.01942398
#> 6                0.9696646                NA 0.04487609

```

```

#>   FPR_G_by_E   FPR_2df power_G power_G_by_E power_2df power_G_pval
#> 1 0.06363027 0.05961152 7.708685   5.559663 10.229024         0
#> 2 0.07568654 0.12659076 5.058862   4.053721  6.603935         0
#> 3 0.08372405          NA 6.006807   5.247079          NA         0
#> 4 0.06697924 0.06229069 7.466928   5.587738 10.047843         0
#> 5 0.05559277 0.05157401 1.602758   6.077606 13.451387         0
#> 6 0.06697924          NA 7.986278   5.568824          NA         0
#>   power_G_by_E_pval power_2df_pval
#> 1                0                0
#> 2                0                0
#> 3                0                NA
#> 4                0                0
#> 5                0                0
#> 6                0                NA

binary_methods <- list(
  fastGWA_GE_naive = fastgwa_bin_naive,
  LEMMA_naive = lemma_bin_naive,
  GENESIS = genesis_bin,
  GEM = gem_bin,
  MRMEGA_GEM = mrmega_gem
)

binary_summary <- summarize_method_list(
  method_results = binary_methods,
  causal_snps = union(binary$causal_main_snps, binary$causal_interaction_snps),
  scenario = binary$result_label,
  seed = bin_inputs$seed,
  mrmega_methods = "MRMEGA_GEM",
  mrmega_pc_num = pc_num_bin
)

binary_summary
#>   method scenario   seed genomic_inflation_factor_G
#> 1 fastGWA_GE_naive binary 100001          0.9732635
#> 2   LEMMA_naive binary 100001          0.9562733
#> 3   GENESIS binary 100001          1.1104008
#> 4     GEM binary 100001          1.0617048
#> 5 MRMEGA_GEM binary 100001          1.5861566
#>   genomic_inflation_factor_G_by_E genomic_inflation_factor_2df   FPR_G
#> 1                0.9986099                0.9905039 0.04353650
#> 2                1.0483800                NA 0.04085733
#> 3                1.0477505                1.0220596 0.05425318
#> 4                1.0996389                1.0691643 0.05358339
#> 5                1.5551675                NA 0.12458138
#>   FPR_G_by_E   FPR_2df power_G power_G_by_E power_2df power_G_pval
#> 1 0.06028131 0.05760214 5.072742   3.432329  8.393848         0
#> 2 0.06430007          NA 4.880383   3.216801          NA         0
#> 3 0.06496986 0.05626256 1.253885   3.196384  7.859967         0
#> 4 0.06430007 0.06095111 4.292676   3.031778  7.171826         0
#> 5 0.12525117          NA 3.484308   3.200806          NA         0
#>   power_G_by_E_pval power_2df_pval
#> 1                0                0

```

#> 2	0	NA
#> 3	0	0
#> 4	0	0
#> 5	0	NA